

Performance evaluation of various functional programming styles

Jan Pustelnik

GFT Poland,
ul. Sterlinga 8a, Lodz, Poland

LambdaDays 2015, 2015-02-26

Outline

1 Introduction

- General
- Benchmarking
- Code

2 Sorting

- MergeSort
- QuickSort
- HeapSort
- Correctness

3 Summary

Intro

This talk is devoted to analysis of how various functional programming styles impact the performance of running programs. The examples presented will be mainly in Haskell, Scala and (!) C++

Hardware

Windows 7 machine. The same for all the tests.

Software – Scala

Scala build config for JMH by Konrad Malawski –
<https://github.com/ktoso/sbt-jmh>.
Scala 2.10.4, Java 1.7.0_71 and JMH 1.5.

Software – Haskell

Criterion library by Bryan O'Sullivan –

<http://www.serpentine.com/criterion/tutorial.html>

GHC 7.8.3, code was compiled with -O option with the standard backend.

Software – C++

Wall clock time, MinGW 4.8.1, GCC version 4.6.3 with -O3 option and run on Windows (yes, it might make a difference).

All fine, but give me the c0de!

Sure, please go to `https://github.com/gosubpl` for the *Source*.

MergeSort

Mergesort is probably the oldest sorting algorithm for computers. Attributed by Knuth to von Neumann around 1945.

Pros:

- Is obviously correct
- Has a nice functional implementation, in fact difficult to implement with mutation
- Works well with lists, tapes, other sequential data structures/media
- $O(n \times \log(n))$ asymptotic – best known for a sorting algorithm that uses comparisons

Cons:

- Maybe slower than quicksort, that will be discovered some fifteen years later

Sources for examples

All programs used as examples are either Public Domain, MIT or Creative Commons by attribution.

- Haskell - <http://en.literateprograms.org/>
- C++ - look for inspiration at http://rosettacode.org/wiki/Sorting_algorithms
- Scala - *Programming in Scala* example improved by Daniel Sobral – <http://stackoverflow.com/questions/2201472/merge-sort-from-programming-scala-causes-stack-overflow>

Top-Down mergesort in Haskell

```
1 mergesort :: (a -> a -> Bool) -> [a] -> [a]
2 mergesort pred [] = []
3 mergesort pred [x] = [x]
4 mergesort pred xs = merge pred (mergesort pred xs1) (
    mergesort pred xs2)
5   where
6     (xs1, xs2) = split xs
```

Top-Down mergesort in Haskell, cntd.

```
1 split :: [a] -> ([a],[a])
2 split xs = go xs xs where
3   go (x:xs) (_:_:zs) = (x:us,vs) where (us,vs)=go xs zs
4   go xs _ = ([],xs)
5
6 merge :: (a -> a -> Bool) -> [a] -> [a] -> [a]
7 merge pred xs [] = xs
8 merge pred [] ys = ys
9 merge pred (x:xs) (y:ys)
10 | pred x y = x: merge pred xs (y:ys)
11 | otherwise = y: merge pred (x:xs) ys
```

Bottom-Up mergesort in Haskell

```
1 mergesort pred [] = []  
2 mergesort pred xs = go [[x] | x <- xs]  
3   where  
4     go xs@(_:_:_ ) = go (pairs xs)  
5     go [xs]         = xs  
6     pairs (x:y:xs) = merge pred x y : pairs xs  
7     pairs xs       = xs
```

You can also do it in C++

It even has two built-in functions for merging: `std::merge` and `std::inplace_merge`

```
1 template<typename RandomAccessIterator, typename Order>
2 void mergesort(RandomAccessIterator first,
3               RandomAccessIterator last, Order order)
4 {
5     if (last - first > 1)
6     {
7         RandomAccessIterator middle = first + (last - first
8         ) / 2;
9         mergesort(first, middle, order);
10        mergesort(middle, last, order);
11        std::inplace_merge(first, middle, last, order);
12    }
13 }
```

You can also do it in C++ – contd.

```
1 template<typename RandomAccessIterator>
2 void mergesort(RandomAccessIterator first ,
3               RandomAccessIterator last)
4 {
5     mergesort(first , last , std::less<typename std::
6         iterator_traits<RandomAccessIterator>::value_type
7         >());
8 }
```

And in Scala

```
1 def msort[T](less: (T, T) => Boolean) (xs: List[T]):  
  List[T] = {  
2   def merge(xs: List[T], ys: List[T], acc: List[T]):  
     List[T] =  
3     (xs, ys) match {  
4       case (Nil, _) => ys.reverse ::: acc  
5       case (_, Nil) => xs.reverse ::: acc  
6       case (x :: xs1, y :: ys1) =>  
7         if (less(x, y)) merge(xs1, ys, x :: acc)  
8         else merge(xs, ys1, y :: acc)  
9     }  
10    val n = xs.length / 2  
11    if (n == 0) xs  
12    else {  
13      val (ys, zs) = xs splitAt n  
14      merge(msort(less)(ys), msort(less)(zs), Nil).  
        reverse  
15    }  
16 }
```


And the performance crown goes to ...

All times in the table below are in milliseconds

Program / Input size	20k	200k	500k	1m	2m
Haskell Bottom-Up	28	562.2	1948	4580	–
Haskell Top-Down	33.5	645.6	2225	5213	–
C++	6.9	–	173.4	355.6	748.8
Scala	9.5	–	411	1035.5	–

The table above gives us some point of reference – no thrills, really.

QuickSort

QuickSort is quick. Discovered by Hoare around 1960. Proven to work around 1969. In the meantime Hoare was busy inventing Hoare logic to use it to prove that quicksort is not only quick but also sorts.

QuickSort – contd.

Pros:

- Is quick
- Apart from imperative mutating implementation has a nice functional one ...

Cons:

- Who knows what it does
- Demands mutation!
- Does not work well with lists, tapes, other sequential data structures/media – requires random access data structure
- $O(n \times \log(n))$ asymptotic – but $O(n^2)$ pessimistic
- ... but unfortunately that functional implementation is not quick!

As previously, credits

- Haskell Functional – <http://en.literateprograms.org/> but almost identical programs can be found in [2] or [3]
- Haskell Imperative – <http://stackoverflow.com/questions/11481675/using-vectors-for-performance-improvement-in-haskell>
- C++ – could not find this anywhere, had to code it, but it was easy, doh!
- Scala – *Scala by Example* by Martin Odersky, except for the ST monadic example – this is from <https://github.com/fpinscala/fpinscala/blob/master/exercises/src/main/scala/fpinscala/localeffects/LocalEffects.scala> (solutions to *Functional programming in Scala* by Paul Chiusano and Rúnar Bjarnason).

Standard mutable quicksort in Scala

Standard, *ugly* and i-am-not-quite-sure-it-is-working implementation of the quicksort algorithm in a language that supports direct mutation – Scala

```
1  def sortQuickTraditional(xs: Array[Int]): Array[Int]
    = {
2    def swap(i: Int, j: Int) {
3      val t = xs(i)
4      xs(i) = xs(j)
5      xs(j) = t
6    }
```

Standard mutable quicksort in Scala – contd.

```
1  def sort1(l: Int, r: Int) {
2      val pivot = xs((l + r) / 2)
3      var i = l
4      var j = r
5      while (i <= j) {
6          while (xs(i) < pivot) i += 1
7          while (xs(j) > pivot) j -= 1
8          if (i <= j) {
9              swap(i, j)
10             i += 1
11             j -= 1
12         }
13     }
14     if (l < j) sort1(l, j)
15     if (j < r) sort1(i, r)
16 }
17 sort1(0, xs.length - 1)
18 xs
```

Standard mutable quicksort in Haskell

One has to use the ST monad.

```
1 stuquick :: [Int] -> [Int]
2 stuquick [] = []
3 stuquick xs = runST (do
4     let !len = length xs
5     arr <- newListArray (0,len-1) xs
6     myqsort arr 0 (len-1)
7     let pick acc i
8         | i < 0      = return acc
9         | otherwise = do
10             !v <- unsafeRead arr i
11             pick (v:acc) (i-1)
12     pick [] (len-1))
```

Standard mutable quicksort in Haskell – contd.

```
1 myqsort :: STUArray s Int Int -> Int -> Int -> ST s ()
2 myqsort a lo hi
3   | lo < hi = do
4       let lscan p h i
5           | i < h = do
6               v <- unsafeRead a i
7               if p < v then return i else lscan p
8                   h (i+1)
9           | otherwise = return i
10      rscan p l i
11      | l < i = do
12          v <- unsafeRead a i
13          if v < p then return i else rscan p
14              l (i-1)
15      | otherwise = return i
```


Standard mutable quicksort in Haskell – contd.

```
1      swap i j = do
2          v <- unsafeRead a i
3          unsafeRead a j >>= unsafeWrite a i
4          unsafeWrite a j v
5      sloop p l h
6      | l < h = do
7          l1 <- lscan p h l
8          h1 <- rscan p l1 h
9          if (l1 < h1) then (swap l1 h1 >>
10             sloop p l1 h1) else return l1
11      | otherwise = return l
12      piv <- unsafeRead a hi
13      i <- sloop piv lo hi
14      swap i hi
15      myqsort a lo (i-1)
16      myqsort a (i+1) hi
17  | otherwise = return ()
```

ST monad mutable quicksort Scala

```
1  def sortQuickMonadicST(xi: Array[Int]): Array[Int] =  
    {  
2      def invert(x: (Int, Int)): (Int, Int) = (x._2, x._1)  
        )  
3      def identify(x: Int, y: Int): Int = y  
4      val arrLen = xi.length  
5      val xiz = xi.zipWithIndex  
6      val xizi = (xiz map invert).toList  
7  
8      type ForallST[A] = Forall[(type λ[S] = ST[S, A])#  
        λ]  
9      def noop[S] = ST[S, Unit]()
```

ST monad mutable quicksort Scala – contd.

```
1  def swap[S](a: STArray[S, Int], i: Int, j: Int): ST[
    S, Unit] = for {
2      x <- a.read(i)
3      y <- a.read(j)
4      _ <- a.write(i, y)
5      _ <- a.write(j, x)
6  } yield ()
```

ST monad mutable quicksort Scala – contd.

```
1  def partition[S](a: STArray[S, Int], l: Int, r: Int,
2    pivot: Int): ST[S, Int] = for {
3    vp <- a.read(pivot)
4    _ <- swap(a, pivot, r)
5    j <- newVar(l)
6    _ <- (l until r).foldLeft(noop[S])((s, i) => for
7      {
8        _ <- s
9        vi <- a.read(i)
10       _ <- if (vi < vp) (for {
11         vj <- j.read
12         _ <- swap(a, i, vj)
13         _ <- j.write(vj + 1)
14       } yield ())
15       else noop[S]
16     } yield ())
17    x <- j.read
18    _ <- swap(a, x, r)
```

ST monad mutable quicksort Scala – contd.

```
1  def qs[S](a: STArray[S, Int], l: Int, r: Int): ST[S  
    , Unit] = if (l < r) for {  
2      pi <- partition(a, l, r, l + (r - l) / 2)  
3      - <- qs(a, l, pi - 1)  
4      - <- qs(a, pi + 1, r)  
5  } yield ()  
6  else noop[S]  
7  
8  def el[S] = for {  
9      arr <- newArr[S, Int](arrLen, 0)  
10     - <- arr.fill(identify, xizi)  
11     - <- qs(arr, 0, arr.size - 1)  
12     sorted <- arr.freeze  
13 } yield sorted
```

ST monad mutable quicksort Scala – contd.

```
1    runST(new ForallST[ImmutableArray[Int]] {  
2        def apply[S] = e1[S]  
3    }).toArray  
4 }
```

It might have not been the easiest to comprehend pieces of code, but maybe it runs fast? Who knows ...

Pure functional quicksort in Haskell

Short, simple and easy to understand.

```
1 qsort1 [] = []  
2 qsort1 (p:xs) = qsort1 lesser ++ [p] ++ qsort1 greater  
3   where  
4       lesser = [ y | y <- xs, y < p ]  
5       greater = [ y | y <- xs, y >= p ]
```

Pure functional quicksort in Scala

As easy as in Haskell :) You can also replace `Array` with `ArrayBuffer` or `Vector`¹

```
1  def sortFunctional(xs: Array[Int]): Array[Int] = {  
2    if (xs.length <= 1) xs  
3    else {  
4      val pivot = xs(xs.length / 2)  
5      Array.concat(sortFunctional(xs filter (pivot >)),  
6                   xs filter (pivot ==), sortFunctional(xs  
7                   filter (pivot <)))  
6    }  
7  }
```

¹`Vector` is immutable and should behave like an `Array` plus have nice amortised complexities.

Pure functional quicksort in C++

Maybe even easier than in Haskell and maybe not. But maybe faster?

```
1 vector<int> funqsort(vector<int> v) {  
2     if (v.size() > 1) {  
3         int pivot = v[0];  
4         vector<int> lesser; vector<int> greater;  
5         std::copy_if(v.begin()+1, v.end(), std::back_inserter  
6             (lesser), std::bind2nd(std::less<int>(), pivot));  
7         std::copy_if(v.begin()+1, v.end(), std::back_inserter  
8             (greater), std::not1(std::bind2nd(std::less<int>  
9             >(), pivot)));  
10        vector<int> result;
```

Pure functional quicksort in C++ – contd.

```
1    vector<int> fql = funqsort(lesser);  
2    vector<int> fkg = funqsort(greater);  
3    std::copy(fql.begin(), fql.end(), std::back_inserter(  
        result));  
4  
5    result.push_back(pivot);  
6  
7    std::copy(fkg.begin(), fkg.end(), std::back_inserter(  
        result));  
8  
9    return result;  
10 } else { return v; }  
11 }
```

Ok, so tell me about the performance ...

All times in milliseconds

Program / Input size	20k	200k	500k	1m	2m
MergeSort Scala	9.5	–	411	1035.5	–
Haskell Imperative	4.2	61.5	161.2	347.7	–
Haskell Functional	18.9	360.4	1072	2747	–
Haskell Fnct. saving	6.9	31.8	515.4	1280	–
StdLibSort C++	1.0	–	29.1	58.4	117.1
C++ Functional	44.5	–	1626	3751	8881
Scala Imperative	1.7	–	54.3	112.7	231.8
Scala Imp. Vector	17.1	–	816.5	1790	4648
Scala Functional	33.46	–	1058	2192	4726
Scala Funct. Vector	22.9	–	865.7	1981	4375
Scala ST Monad	138.3	–	5535	13428	35290

HeapSort

HeapSort is very easy to implement if you have a Heap or PriorityQueue data structure at hand. Benchmarking it gives you a chance to compare performance of various PriorityQueue implementations.

Pros:

- Very simple to implement if you have a PriorityQueue
- $O(n \times \log(n))$ asymptotic

Cons:

- If your library supports PriorityQueues, probably it has a standard sorting algorithm too
- Slower than QuickSort and not stable like MergeSort

Sources for examples

- Haskell – `Data.Heap` from the `heap` package (leftist trees from Okasaki by Edward Kmett) and `Data.PQueue` from the `pqueue` package (binomial heaps).
- Scala – one imperative implementation using `mutable.PriorityQueue` from the standard library, and one purely functional implementation using `scalaz.Heap` that implements leftist trees (also by Edward Kmett)

Heap sorts in Haskell

```
1 import qualified Data.Heap as DH — from heap package
2 import qualified Data.PQueue.Min as DPQMin — from
   pqueue package
3
4 — heapsort
5 hsort :: [Int] -> [Int]
6 hsort xs = DH.toAscList (DH.fromList xs :: DH.MinHeap
   Int)
7
8 — another heapsort
9 hpqsort :: [Int] -> [Int]
10 hpqsort xs = DPQMin.toAscList (DPQMin.fromList xs)
```

Mutable heap sort in Scala

```
1  // a bit mutable heapsort using the StdLib mutable.  
   PriorityQueue  
2  def sortHeapPQ(xs: Array[Int]): Array[Int] = {  
3    val ord = implicitly[Ordering[Int]].reverse  
4    val lst = ListBuffer[Int]()  
5    val pq: PriorityQueue[Int] = new PriorityQueue[Int]  
      ]()(ord) ++ xs  
6    while (pq.size > 0) {  
7      val elem = pq.dequeue()  
8      lst += elem  
9    }  
10   lst.toArray  
11 }
```

Pure immutable heap sort in Scala

```
1  def sortHeapLeftist(xs: Array[Int]): Array[Int] = {
2    import scalaz._
3    import Scalaz._
4    def poorMansHeapFold(h: Heap[Int]): List[Int] = {
5      def heapFoldLeftAccum(accum: List[Int], h: Heap[
6        Int]): List[Int] = {
7        if (h.size == 0) {
8          accum.reverse
9        } else {
10         val (head, tail) = h.uncons.get
11         heapFoldLeftAccum(head :: accum, tail)
12       }
13     }
14     heapFoldLeftAccum(Nil, h)
15   }
16   poorMansHeapFold(Heap.fromData(xs.toList)).toArray
17 }
```


But, I want the numbers!

All times in milliseconds

Program / Input size	20k	200k	500k	1m	2m
MergeSort Scala	9.5	–	411	1035.5	–
Haskell QSort Funct.	18.9	360.4	1072	2747	–
Scala QSort Imp.	1.7	–	54.3	112.7	231.8
Scala PriorityQueue	17.1	–	330.8	834.1	–
Scala ScalazHeap	135.5	–	6778	15363	–
Scala QuickST	138.3	–	5535	13428	35290
Haskell PQueue	24.8	506.4	1735	4108	–
Haskell Heap	40.3	740.3	2475	5606	–

QuickCheck

QuickSort is quick. Discovered by Hoare around 1960. Proven to work around 1969. In the meantime Hoare was busy inventing Hoare logic to use it to prove that quicksort is not only quick but also sorts. . .

QuickCheck to the rescue!

```
1 — usage: quickCheckN 10000 prop_qsort_isOrdered
2
3 isOrdered (x1:x2:xs) = x1 <= x2 && isOrdered (x2:xs)
4 isOrdered _ = True
5
6 prop_qsort_isOrdered :: [Int] -> Bool
7 prop_qsort_isOrdered = isOrdered . qsort1
8
9 quickCheckN n = quickCheckWith \$ stdArgs { maxSuccess
    = n }
```

- Imperative constructs win in terms of performance
- Haskell seems to be faster than Scala, C++ for *pure functional* constructs
- Garbage is a problem in functional languages (in 1960's it was called *consing* – but who uses Lisps anymore)
- Scalaz is a library, Haskell is a compiler
- Pragmatic functional languages accept that mutation can be a fact of life
- One should isolate mutation, Monads not perfect – imperative code even less readable
- If it has functional interface, I can simply test it with randomized tests, don't have to worry what's inside

Bibliography

-  Okasaki C., *Purely functional data structures*, Cambridge University Press, 1999
-  Rabhi F., Lapalme G., *Algorithms, A Functional Programming Approach*, Addison-Wesley, 1999
-  Bird R., *Introduction to Functional Programming using Haskell*, Prentice-Hall, 1998
-  Chusano P., Bjarnason R., *Functional Programming in Scala*, Manning, 2014
-  Odersky, M., Spoon, L., Venners, B. *Programming in Scala, 2nd ed.*, Artima, 2011